

Context-Aware Deadline Scheduling Under Memory Contention for Embedded Real-Time Systems

Eren Karadağ* and Umut Kalman†

*Computer Engineering Department, Hacettepe University, Ankara, Türkiye

†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye

E-mails: *erenkaradag@hacettepe.edu.tr, †umutkalman@hacettepe.edu.tr

Abstract—Modern embedded systems are increasingly required to handle complex, mixed-criticality workloads where computation-bound control loops are run together with data-intensive processing tasks. In these constrained environments, memory contention has come out as a critical factor threatening real-time determinism. Although standard scheduling algorithms, such as Rate Monotonic (RM) and Earliest Deadline First (EDF), optimize primarily for CPU utilization and deadline compliance, they often fail to account for the overhead introduced by memory interference. This project presents a context-aware scheduling strategy designed to reduce these penalties in real-time embedded systems.

We propose a lightweight, user-space scheduler wrapper implemented on FreeRTOS that extends the classic EDF algorithm with a "Slack-Aware" heuristic. This mechanism dynamically evaluates the memory intensity of pending tasks and introduces a penalty cost model with metrics to predict contention. With task slack times in mind, the scheduler reorders execution sequences to minimize consecutive memory-heavy operations without violating hard deadlines. To validate the effectiveness of this online heuristic, we first develop an offline Ant Colony Optimization (ACO) model that searches for a low-cost near-optimal reference schedule for a given task set. Then, we deploy our online scheduler wrapper in an application running on a microcontroller with a single-core processor to examine the custom scheduler we developed.

Experimental results compare the proposed slack-aware EDF wrapper scheduler against standard pure EDF scheduling as baseline, using the ACO solution as a performance baseline. The study demonstrates that the proposed context-aware scheduler significantly reduces simulated memory-contention penalties compared to traditional EDF, while maintaining similar deadline-miss behavior and the determinism required for embedded real-time applications.

Index Terms—Real-time embedded systems, memory contention, Earliest Deadline First (EDF) scheduling, slack-aware scheduling, Ant Colony Optimization (ACO).

I. INTRODUCTION

Real-time systems are foundational to safety-critical infrastructure, including automotive control units, defense systems, robotics, avionics, and medical devices. In these critical environments, timing predictability is naturally tied to overall system safety and reliability; a missed deadline in a critical component, such as a braking controller or flight control unit, can lead to unstable operation, severe physical damage, or even life-threatening consequences. As modern embedded systems evolve, they are increasingly required to handle complex, mixed-criticality workloads that run control loops alongside data-intensive processing tasks. With

platforms becoming more memory-intensive, shared memory contention has emerged as a critical threat to real-time determinism.

While classical scheduling theory provides strong guarantees under ideal assumptions, real-world embedded environments are significantly more complex. Standard scheduling algorithms, such as Rate Monotonic (RM) and Earliest Deadline First (EDF), are highly effective at optimizing CPU utilization and deadline compliance. However, they typically fail to account for the hidden execution overheads introduced by memory interference. This oversight creates a distinct gap between theoretical schedulability and observed runtime behavior. In practice, engineers frequently encounter systems that are proven schedulable on paper but still suffer from jitter, unpredictable latency, and sporadic deadline misses when deployed due to this hidden resource interference.

To address these challenges, this paper introduces a context-aware scheduling strategy designed to reduce memory contention penalties in real-time embedded systems without requiring major system redesigns or expensive hardware upgrades. We propose a lightweight, user-space scheduler wrapper implemented on FreeRTOS that extends the classic EDF algorithm with a dynamic slack-aware heuristic. This mechanism evaluates the memory intensity of pending tasks at runtime and utilizes a penalty cost model to predict contention. By leveraging available task slack times, the scheduler intelligently reorders execution sequences to separate consecutive memory-heavy operations, thereby minimizing interference while strictly maintaining hard deadlines.

To thoroughly validate the effectiveness of this approach, the project pairs the online heuristic with an offline Ant Colony Optimization (ACO) model, which generates an offline near-optimal reference schedule to serve as a performance baseline. We will deploy the model on a single-core STM32L4-series microcontroller to examine the custom scheduler's real-world performance, comparing it against standard static priority and pure EDF scheduling methodologies. Ultimately, the goal is to move one step closer to real-time systems that behave in the field as predictably as they do in theory, improving safety,

trust, and efficiency across embedded computing domains.

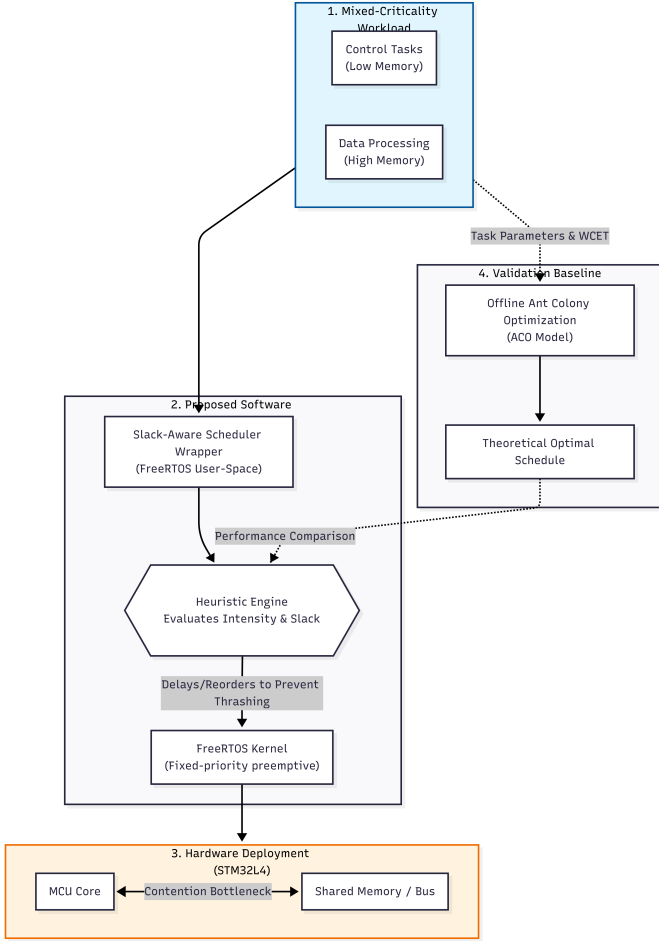


Fig. 1. Proposed custom scheduler wrapper implementation flow

II. RELATED WORK

EDF remains the classical baseline for uniprocessor real-time scheduling because, under ideal assumptions, it is optimal in processor utilization [1]. In practice, however, those assumptions hide preemption and memory-system overheads, so timing behavior can deviate from theoretical guarantees.

Prior research addressed memory interference through system-level control and scheduling structure. OS-level methods such as MemGuard and bank-aware allocation improve isolation by regulating bandwidth and memory placement [2], [3]. PREM-style and memory-centric approaches make memory phases scheduler-visible to improve predictability [4]–[6], but often require explicit phase boundaries and stronger runtime support.

Another line of work extends EDF analysis by modeling cache-related preemption delay (CRPD), showing that cache effects can significantly affect schedulability [7]. Slack reclaiming studies also show that online slack can improve EDF behavior [8], though they mainly target soft real-time optimization rather than memory-aware hard real-time ordering.

Finally, online/offline studies with preemption penalties show that no universally optimal online policy exists in the general case, while offline optimization can provide stronger references [9]. Motivated by this, our work combines a lightweight online EDF wrapper with an offline optimization baseline, using memory intensity and slack jointly without heavy kernel changes or specialized hardware.

III. PROBLEM DEFINITION

A. System / Application Description

The target application is a mixed-critical embedded real-time system, representative of domains such as automotive electronic control units (ECUs) and mission-critical defense platforms. In this setting, the system must execute two workload classes concurrently:

- **Control tasks (low memory intensity):** high-frequency, hard real-time activities for actuation, sensing, and motor-control loops. These tasks are primarily computation-bound and require strict deadline compliance.
- **Data-intensive tasks (high memory intensity):** lower-frequency but high-bandwidth operations, such as signal processing, image filtering, and cryptographic payload handling. These tasks require frequent access to shared memory structures.

When these workload classes are interleaved, memory-intensive executions can increase bus and cache interference, which appears as jitter and unpredictable delay in critical control jobs.

In this study, the representative workload contains four periodic tasks:

- τ_1 (motor control, low memory intensity): a high-frequency hard real-time control task (e.g., PID loop), primarily computation-bound.
- τ_2 (sensor data acquisition, low memory intensity): periodic ADC sampling and filtering used by the control pipeline.
- τ_3 (cryptographic encryption, high memory intensity): lower-frequency telemetry encryption with heavy memory access.
- τ_4 (vision processing, high memory intensity): medium-high priority vision processing task with heavy memory access.

When these tasks are interleaved, memory-intensive executions can increase cache and bus interference, introducing jitter for time-critical control jobs.

1) System Context and Processor Model:

The scheduler wrapper runs on FreeRTOS over an ARM Cortex-M4 STM32L4-series microcontroller. Because execution is single-core, concurrency is realized by preemption and context switching, so contention appears as sequential interference between jobs with different memory-access profiles.

2) Assumptions and Task Model:

The task model uses the tuple (C_i, T_i, D_i, M_i) for each periodic task τ_i , where C_i is isolated WCET, T_i is period,

TABLE I
COMPARISON OF THE MOST RELEVANT STUDIES AND THE POSITION OF THIS WORK.

Work	Year	Multi-core	EDF	Mem/Cache Aware	Slack	Online	Offline Oracle	Heavy Support
Liu & Layland [1]	1973		✓			✓		
MemGuard [2]	2013	✓		✓		✓		✓
PALLOCC [3]	2014	✓		✓		✓		✓
PREM [4]	2011	✓		✓				✓
MCS-RTOS [5]	2019	✓		✓		✓		✓
FP-MCS [6]	2020	✓		✓		✓		✓
EDF+CRPD [7]	2013		✓	✓				
Slack reclaiming [8]	2005		✓		✓	✓		✓
Online/Offline CRPD [9]	2018		✓	✓		✓	✓	
This Work	2026		✓	✓	✓	✓	✓	

D_i is deadline, and $M_i \in [0, 1]$ is normalized memory intensity. Tasks are independent and non-blocking. Baseline EDF schedulability is assumed under isolation:

$$\sum_{i=1}^4 \frac{C_i}{T_i} \leq 1.$$

B. Objectives

- 1) Implement standard EDF scheduling as baseline
- 2) Minimize contention penalties by ordering executions to reduce cumulative memory-interference overhead.
- 3) Maintain real-time guarantees by targeting zero deadline misses for theoretically schedulable task sets.
- 4) Establish a search-based offline reference schedule using ACO for comparison with online scheduling policies.

C. Constraints

- 1) **Timing:** Missing any absolute deadline d_j is treated as a critical failure.
- 2) **Hardware:** Single-core shared-memory execution implies sequential preemption penalties; total CPU utilization must remain at or below 100%.
- 3) **Runtime overhead:** The online FreeRTOS wrapper must remain lightweight so decision overhead does not exceed contention-related savings.

D. Mathematical Formulation (Optimization Track)

Let $\mathcal{J} = \{j_1, j_2, \dots, j_N\}$ denote all released jobs in one hyperperiod. Define binary decision variable

$$x_{a,b} = \begin{cases} 1, & \text{if } j_b \text{ immediately follows } j_a, \\ 0, & \text{otherwise.} \end{cases}$$

The pairwise memory penalty is

$$P(j_a, j_b) = \alpha M_a M_b.$$

The objective is

$$\min \sum_{a=1}^N \sum_{b=1}^N x_{a,b} P(j_a, j_b).$$

a) Sequence Integrity:

$$\sum_{b=1, b \neq a}^N x_{a,b} = 1 \quad \forall a \in \mathcal{J},$$

$$\sum_{a=1, a \neq b}^N x_{a,b} = 1 \quad \forall b \in \mathcal{J}.$$

b) Deadline Compliance:

$$f_b = s_b + C_b + \sum_{a=1}^N (x_{a,b} P(j_a, j_b)),$$

$$f_b \leq d_b \quad \forall b \in \mathcal{J}.$$

c) Online Slack Proxy:

$$Slack_j(t) = d_j - t - C_{rem,j}(t).$$

High-memory tasks are deferred only when $Slack_j(t) > 0$. This formulation defines the offline sequencing objective. Since solving the exact combinatorial problem can become expensive as the number of jobs grows, we use ACO as a search-based metaheuristic approximation rather than an exact ILP solver.

IV. SYSTEM ARCHITECTURE AND DESIGN

A. Hardware/Software Architecture

The implementation combines a resource-constrained embedded platform with a lightweight scheduling layer:

- **Hardware platform:** STM32L4-series microcontroller (ARM Cortex-M4), unified memory bus, and on-chip SRAM.

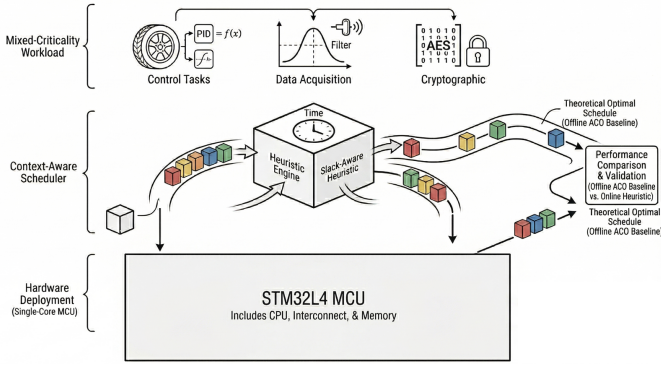


Fig. 2. Tasks running concurrently on MCU

- **Operating system:** FreeRTOS with baseline preemptive tick scheduling.
- **Scheduler wrapper:** custom user-space C/C++ module that inspects ready tasks and adjusts execution order based on slack-aware contention heuristics.

B. Dataflow and System Model

System execution starts with periodic hardware-timer interrupts that release jobs into the FreeRTOS ready queue. Instead of always dispatching the earliest-deadline task immediately, the wrapper evaluates the memory intensity of the running and pending jobs. When predicted pairwise contention is high, the wrapper checks dynamic slack and defers the memory-intensive job only if timing slack remains positive.

C. Timing and Execution Model

The system follows a periodic, event-driven model implemented with `vTaskDelayUntil()`. The base dispatching rule is EDF with dynamic deadlines. The wrapper applies bounded, slack-aware reordering to reduce contention without violating real-time constraints.

D. Model of Computation

The design is modeled as a discrete-event system with synchronous periodic releases. Scheduling events are triggered by job releases, job completions, and contention-aware wrapper checks.

E. Assumptions and Design Limitations

The current design assumes accurate offline profiling of WCET (C_i) using ARM DWT counters. It also treats memory intensity (M_i) as a static task-level characteristic; fine-grained intra-task phase variation is not explicitly modeled.

F. Resource Model

The proposed scheduler is designed under strict embedded resource limits of a single-core STM32L4-series platform. These limits directly motivate the contention-aware design.

1) **Memory Usage (RAM and Buffers):** The platform provides limited SRAM (typically 128–192 KB). A fixed portion is reserved by FreeRTOS, including task control blocks (TCBs) and task stacks. Control-oriented tasks (τ_1, τ_2) use small local buffers, whereas the cryptographic task (τ_3) and vision processing task (τ_4) use larger contiguous buffers (approximately 4–8 KB).

Since τ_3 and τ_4 drive high memory traffic, the scheduler wrapper must remain lightweight. Runtime dynamic allocation is avoided, and task metadata lookup is designed to be constant-time ($O(1)$), preventing additional memory pressure.

2) **CPU Utilization:** Execution runs on a single ARM Cortex-M4 core with no GPU or co-processor offload. The baseline task set is assumed to be schedulable in isolation with total utilization below 100%.

Because all tasks share one core, scheduling overhead must be tightly bounded. Slack computation and queue reordering are implemented with low computational cost (targeting near- $O(n)$ with respect to ready-queue size) so the wrapper does not introduce avoidable deadline risk.

V. METHODOLOGY

The project methodology is divided into an offline optimization pipeline (for benchmarking) and an online algorithm implementation. All project code and data will be version-controlled and hosted in the AtomicRadars GitHub organization repository for transparency and evaluation.

A. Optimization Pipeline (Optimization Track)

The optimization pipeline proceeds in three stages:

- 1) **Workload generation:** Define the task-set parameters (C_i, T_i, D_i, M_i) from STM32 profiling measurements and generate all jobs over one hyperperiod.
- 2) **Offline ACO solving:** Run a Python-based Ant Colony Optimization solver using the sequencing formulation in Section III-D. Candidate schedules are constructed iteratively, and pheromone updates favor job-to-job or segment-to-job transitions that lead to lower schedule cost.
- 3) **Reference extraction:** Use the best schedule found by ACO as a search-based near-optimal reference for comparing online policies. Since ACO is a metaheuristic, this reference is not claimed to be an exact mathematical optimum.

B. Offline ACO Solver Formulation

The offline ACO solver is used as a reference optimizer for the scheduling problem. In the non-preemptive mode, each ant constructs a complete job ordering. In the preemptive mode, each ant constructs an execution trace composed of smaller job segments. This distinction is useful because the embedded FreeRTOS implementation is preemptive, while the non-preemptive model helps expose the effect of long memory-heavy jobs blocking short-deadline jobs.

Let τ_{ab} denote the pheromone value associated with selecting job or segment j_b immediately after j_a . Let η_{ab} denote

the heuristic desirability of this transition. The heuristic term reflects local scheduling information such as deadline urgency, slack, remaining execution time, and memory-transition cost. For a candidate set $\mathcal{A}$, the probability of selecting j_b after j_a is given by

$$p_{ab} = \frac{\tau_{ab}^{\alpha_{aco}} \eta_{ab}^{\beta_{aco}}}{\sum_{k \in \mathcal{A}} \tau_{ak}^{\alpha_{aco}} \eta_{ak}^{\beta_{aco}}}.$$

Here, α_{aco} controls the influence of pheromone history, while β_{aco} controls the influence of the heuristic desirability. The denominator normalizes the scores of all candidate jobs and converts them into probabilities.

After each iteration, pheromone evaporation is applied:

$$\tau_{ab} \leftarrow (1 - \rho)\tau_{ab},$$

where ρ is the evaporation rate. This prevents the search from converging too early to a poor sequence. After evaluating a constructed schedule S , transitions used in good schedules receive additional pheromone:

$$\Delta\tau_{ab} = \frac{Q}{Cost(S)}, \quad \tau_{ab} \leftarrow \tau_{ab} + \Delta\tau_{ab}.$$

Thus, lower-cost schedules deposit more pheromone and make their transitions more likely to be selected in later iterations.

The schedule cost combines real-time correctness and memory behavior:

$$Cost(S) = w_{miss}M_{miss} + w_{late}L + w_{cont}P_{cont} + w_{smooth}P_{smooth} + P_{extra} \quad (1)$$

Here, M_{miss} is the number of deadline misses, L is total lateness, P_{cont} is the discrete contention penalty, P_{smooth} is the continuous smooth memory-transition penalty, and P_{extra} represents optional preemptive-mode costs such as response-time or segment-level costs. The deadline-miss weight w_{miss} is intentionally dominant, so memory penalties are optimized only after real-time correctness is prioritized.

C. Algorithm Description

The core online policy is a slack-aware wrapper around EDF. At each dispatch point, the wrapper estimates pairwise memory contention and applies deferral only when sufficient timing slack is available.

Algorithm 1: Slack-Aware Scheduling Heuristic

```

1: Input: ready queue  $Q$ , current task  $T_{curr}$ , system time  $t$ 
2: Output: next task  $T_{next}$ 
3: Sort  $Q$  by earliest absolute deadline (EDF order)
4:  $T_{cand} \leftarrow Q[0]$ 
5:  $Penalty \leftarrow \alpha \cdot M_{curr} \cdot M_{cand}$ 
6: if  $Penalty > Threshold$  then
7:    $Slack \leftarrow d_{cand} - t - C_{cand}$ 
8:   if  $Slack > SafetyMargin$  then
9:     Find  $T_{alt} \in Q$  such that  $M_{alt}$  is low and  $T_{alt}$  remains schedulable
10:   if  $T_{alt}$  exists then
11:     return  $T_{alt}$ 
```

```

12:   end if
13: end if
14: end if
15: return  $T_{cand}$ 
```

VI. EXPERIMENTAL SETUP

A. Platform and Tools

The experimental environment combines an embedded target platform with offline optimization tooling:

- **Hardware:** STM32L452 Nucleo-64 evaluation board.
- **Operating system and toolchain:** FreeRTOS, CMake, and STM32Cube extension in VSCode IDE for build, deployment, and debugging. STM32CubeMX for auto-generated C peripheral codes, and C++23 for application code.
- **Profiling support:** ARM Data Watchpoint and Trigger (DWT) cycle counter for microsecond-level timing measurements.
- **Offline optimizer:** Python with NumPy for the ACO implementation.

B. Datasets / Workloads

Evaluation uses the synthetic mixed-criticality four-task workload defined in Section III-A. To emulate memory pressure, the cryptographic task executes matrix-style memory operations and AES-like transformations whereas the vision processing task imitates image filtering operations on a large contiguous SRAM buffer, while the control task runs computation-dominant floating-point updates with low memory intensity. For offline analysis, additional workload profiles are also used in the simulator. The *memory-stress* profile introduces additional memory-heavy and low-memory filler tasks to emphasize transition-ordering effects. The *high-utilization* profile increases WCET values to evaluate scheduler behavior under tighter timing pressure.

C. Evaluation Metrics

Performance is assessed using the following metrics:

- **Deadline miss ratio (DMR):** percentage of released jobs that violate D_i .
- **Total lateness:** cumulative amount of time by which missed jobs exceed their absolute deadlines.
- **Task jitter:** response-time variance of the critical control task (τ_1).
- **Total contention penalty:** aggregate threshold-based memory-interference delay over one hyperperiod. This metric captures hard memory-heavy back-to-back transitions.
- **Smooth memory penalty:** continuous transition score computed from consecutive task memory intensities. Unlike the discrete contention penalty, it assigns a gradual cost to all transitions.
- **ACO schedule cost:** weighted sum of deadline misses, total lateness, contention penalties, smooth memory penalty, and optional preemptive-mode segment or response-time costs.

- **Context-switch overhead:** execution cost of Algorithm 1, measured in DWT clock cycles.

VII. SYSTEM-LEVEL CONSIDERATIONS

- **Real-time constraints and latency:** The proposed wrapper targets latency by reducing jitter caused by memory interference. To preserve hard real-time safety, the policy falls back to baseline EDF whenever available slack is insufficient for safe deferral.
- **Energy efficiency:** Although not the primary optimization objective, lowering cache and bus thrashing can reduce energy usage by decreasing high-cost memory transactions.
- **Scalability:** The current heuristic has approximately $O(n)$ complexity in the ready-queue size. For larger task sets, future work includes constant-time ($O(1)$) bitmap-based admission checks and lightweight hardware-assisted scheduling primitives.
- **Reliability and fault tolerance:** By aligning online behavior with offline near-optimal ordering trends, the approach improves timing consistency under heavy memory pressure and reduces the risk of unstable runtime behavior in safety-critical deployments.

VIII. RESULTS AND EVALUATION

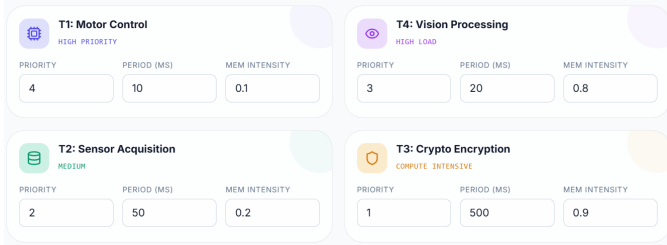


Fig. 3. Tasks' parameters

Fig.3 represents the parameters used for the online benchmark across three different scheduler solutions: Native, EDF, and the proposed Context-Aware scheduler (high priority number = higher priority). As shown in the table, τ_3 and τ_4 have high memory intensities whereas τ_1 and τ_2 have high priorities. In order to simulate heavy workload without any external sensors for proof-of-concept, τ_4 has a higher priority than τ_2 .

A. Performance Results

Fig.4 through Fig.7 illustrate the performance of our proposed context-aware scheduler wrapper across various tasks with different workloads. The contention penalty remains the same across tasks, but other evaluation metrics such as jitter and DMR vary.

Fig.4 and Fig.5 represent the performance when running low memory but high priority tasks. DMR values are 2% and 4% respectively which are higher than high memory intensity tasks τ_3 and τ_4 . Since our scheduler takes memory intensities into account, tasks with high memory intensities get prioritized and

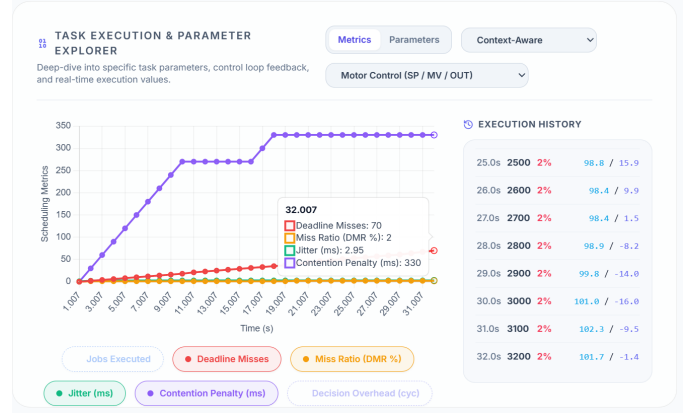


Fig. 4. Context-Aware scheduler wrapper performance for τ_1

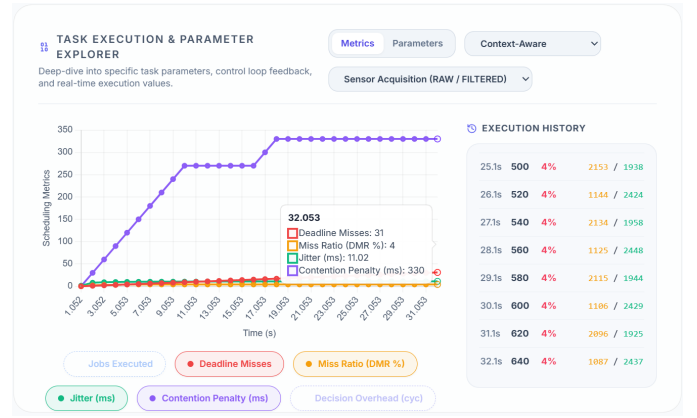


Fig. 5. Context-Aware scheduler wrapper performance for τ_2

thus not deferred. Also, no jobs of task τ_3 are missed under the proposed scheduler.

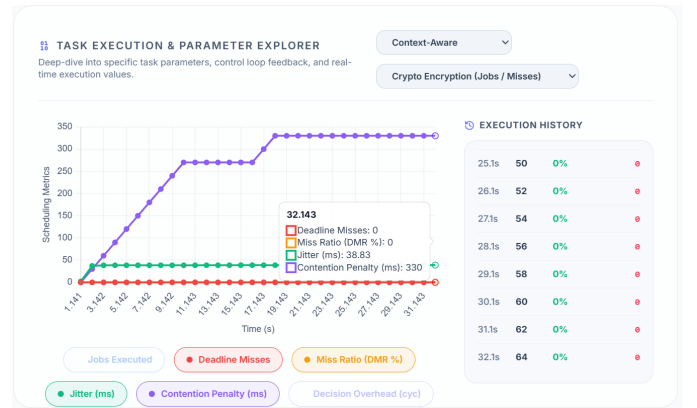


Fig. 6. Context-Aware scheduler wrapper performance for τ_3

For response-time jitter, Welford's Algorithm is used in our study.

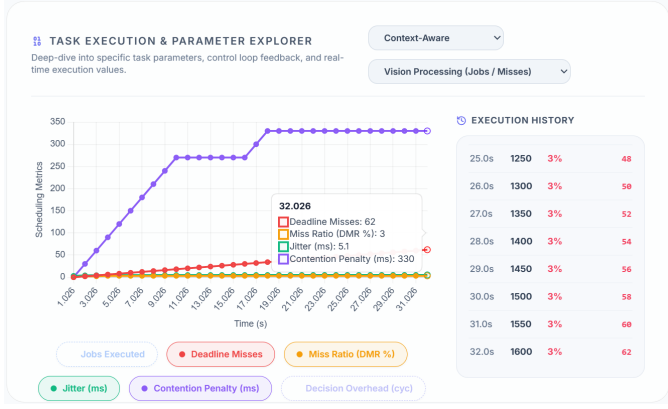


Fig. 7. Context-Aware scheduler wrapper performance for τ_4

Welford's Algorithm

At the n -th sample:

$$\begin{aligned}\delta_n &= x_n - \mu_{n-1} \\ \mu_n &= \mu_{n-1} + \frac{\delta_n}{n} \\ M_{2,n} &= M_{2,n-1} + \delta_n \cdot (x_n - \mu_n) \\ s_n &= \sqrt{\frac{M_{2,n}}{n-1}}, \quad n > 1\end{aligned}$$

Variables: n is the sample count, x_n is the current response time, μ_n is the running mean, δ_n is the mean difference, $M_{2,n}$ is the accumulated squared deviation, and s_n is the response-time jitter.

The jitters of the high memory intensity tasks τ_3 and τ_4 are 38.83 and 5.1 ms and are higher than the low memory intensity tasks on average. Although the jitter is higher for high memory intensity tasks, a smaller number of jobs are missed compared to the low memory intensity tasks.

B. Comparative Analysis

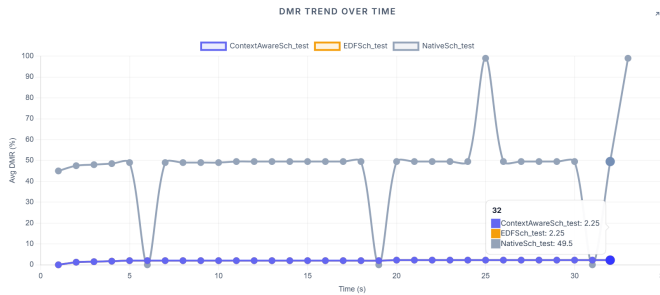


Fig. 8. DMR trend over time graph with three schedulers

Fig.8 displays the DMR trend over time for the duration of the benchmark. In this case, the benchmark is run for 32 seconds for three different schedulers and the DMR is measured. As can be seen, the DMR values of EDF and Context-Aware schedulers are virtually the same, while the Native scheduler even misses all of its jobs sometimes. EDF and Context-Aware schedulers outperform FreeRTOS's

own native scheduler under the four-mixed-criticality-task-workload scenario.

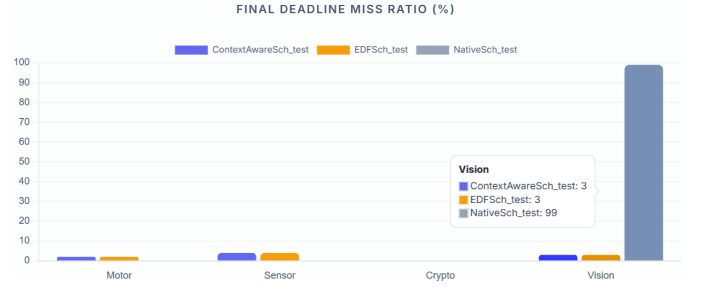


Fig. 9. Final DMR values for four tasks of three schedulers

Fig.9 illustrates the DMR values for the four different tasks that have been benchmarked in this study. The native scheduler missed a tiny number of jobs in τ_1 , whereas missed 100% of its jobs in τ_2 and τ_3 . The tasks τ_2 and τ_3 are completely starved under the native scheduler policy due to memory contention.

The baseline of our study, EDF, performed essentially the same as the Context-Aware scheduler. While DMR values remain virtually identical, the total memory contention penalty serves as the key distinguishing factor.



Fig. 10. Task jitter, total contention penalty, and overhead values for different schedulers

Although DMR values are quite similar for EDF and Context-Aware scheduler, our proposed wrapper has less than half of the contention penalty compared to EDF while having the same task jitter value.

C. Offline ACO Reference Results

In addition to the hardware-in-the-loop experiments, the offline ACO solver was used to evaluate search-based reference schedules under different workload profiles. These experiments do not replace the embedded measurements; rather, they provide an offline comparison point for understanding how much scheduling order can affect deadline behavior and memory-transition penalties.

Table II shows that the benefit of ACO depends on the workload regime. In the memory-stress profile, EDF and Context-Aware already avoid deadline misses, and the Context-Aware scheduler achieves the lowest smooth memory penalty and total cost among online policies. In the high-utilization profile, ACO finds a lower-cost dispatch trace by reducing both deadline misses and total lateness compared to EDF and Context-Aware. This result supports the role of ACO as a search-based reference: it helps reveal when global dispatch ordering provides additional benefit beyond local online heuristics.

TABLE II
OFFLINE ACO REFERENCE RESULTS UNDER REPRESENTATIVE WORKLOAD PROFILES.

Workload	Scheduler	Misses	Lateness (ms)	Contention	Smooth Penalty	Cost
Memory Stress / Preemptive	RMS	6	412	14	15.50	10.12M
Memory Stress / Preemptive	EDF	0	0	14	14.82	226.30
Memory Stress / Preemptive	Context-Aware	0	0	14	14.29	215.80
Memory Stress / Preemptive	ACO	0	0	14	14.58	221.60
High Utilization / Preemptive	RMS	1	10	40	23.36	1.10M
High Utilization / Preemptive	EDF	2	13	38	23.03	2.13M
High Utilization / Preemptive	Context-Aware	2	9	36	22.40	2.09M
High Utilization / Preemptive	ACO	1	6	36	22.38	1.06M

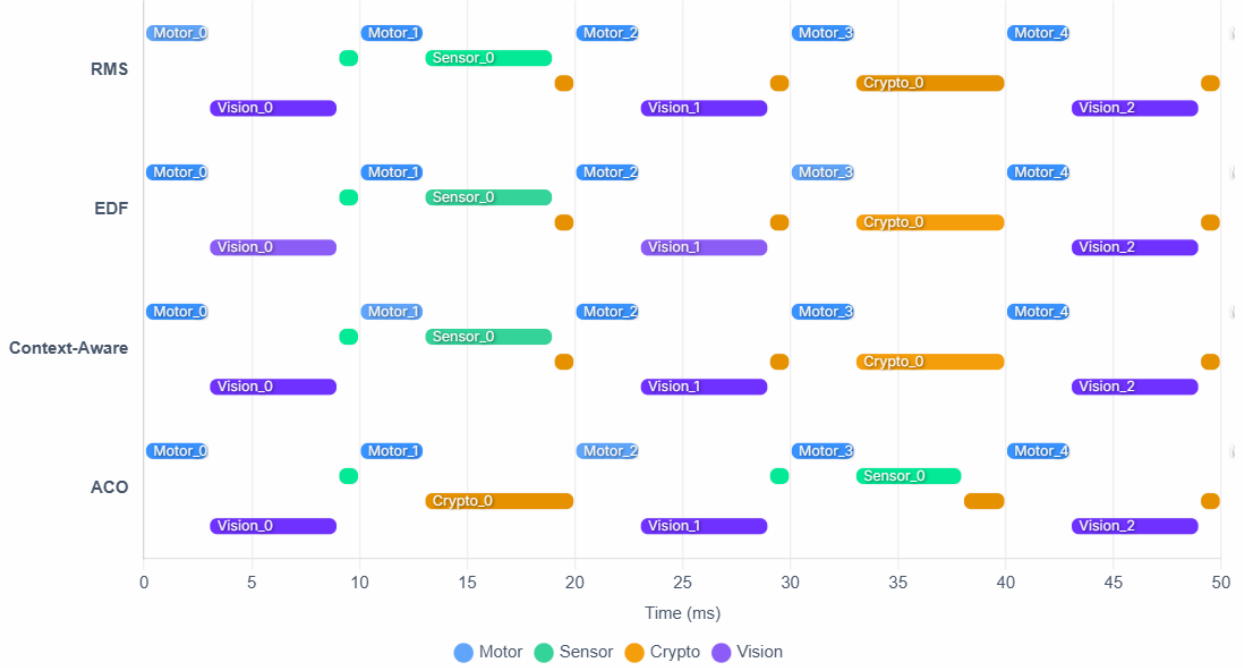


Fig. 11. Scheduler comparison Gantt chart for the preemptive high-utilization workload. The Gantt chart is shown as a representative visualization from the same workload profile; numerical values are reported in Table II. The figure shows the first 50 ms of execution for RMS, EDF, Context-Aware, and offline ACO. Each colored block represents a scheduled job or execution segment, making the dispatch-order differences across schedulers visually comparable.

Figure 11 provides a visual comparison of the dispatch traces produced by RMS, EDF, Context-Aware, and offline ACO under the preemptive high-utilization workload. Even within the first 50 ms, differences in execution ordering can be observed, especially in how short-period jobs are interleaved with memory-intensive tasks. This supports the numerical results by showing that scheduler quality depends not only on utilization, but also on the ordering of task executions.

The offline ACO results should therefore be interpreted as a validation and analysis tool rather than a directly deployable online scheduler. The online Context-Aware Scheduler remains lightweight and practical for FreeRTOS, while ACO provides a stronger but offline comparison point for schedule quality.

D. Discussion

In this paper, a scheduling method which takes memory contention and task slack into consideration is proposed and compared against FreeRTOS's own scheduler and EDF as baseline. We designed our scheduler to extend the classic EDF algorithm with a "Slack-Aware" heuristic that also reorders execution sequences to minimize consecutive memory-heavy operations without violating hard deadlines on embedded systems.

We have successfully built upon EDF and introduced slack and memory contention awareness. With the same task deadlines and priorities, Context-Aware Scheduler has less total contention penalty compared to EDF while maintaining similar DMR and jitter. The average decision overhead is higher than

EDF which is acceptable, since integrating context awareness introduces more computation cycles.

The offline ACO results further clarify the behavior of the online wrapper. In less constrained preemptive workloads, EDF already performs well because preemption prevents long jobs from blocking urgent releases. Therefore, ACO provides only small improvements in smooth memory-transition cost. However, under tighter high-utilization workloads, the ACO reference finds a lower-cost dispatch trace with fewer deadline misses and lower total lateness. This indicates that global sequencing can still improve performance when local EDF-style decisions become insufficient.

In order to benchmark the schedulers, we used USART2 peripheral to print metrics onto the serial port on our PC. This approach caused the observer effect on the single-core STM32 MCU. Since UART interrupts stall the pipeline and compete for the exact resources as our tasks, it introduced its own hardware contention.

E. Reproducibility

To ensure the reproducibility of our findings, all source code, configuration scripts, and the full experimental testing suite are publicly available in the AtomicRadars GitHub organization repository.

The repository is logically structured into three primary components to support both embedded and systems-level validation:

- **Embedded Firmware Core:** Written in C11 and C++23, the firmware targets the STM32L452XX microcontroller. It operates on FreeRTOS and is configured via STM32CubeMX. Build orchestration is fully managed by CMake (v3.22+), compiling via the GNU Arm Embedded Toolchain (arm-none-eabi-gcc).
- **Interactive Web GUI & Test Lab:** A Python 3.9+ FastAPI full-stack dashboard that acts as a performance lab. It supports live parameter customization (e.g., penalty weight α , safety margins, memory thresholds), triggers automated hardware benchmarking by programmatically updating `TaskConfig.h`, builds via CMake, flashes via `STM32_Programmer_CLI`, and plots real-time telemetry (DMR%, Welford's jitter, contention penalties).
- **Offline Optimization Pipeline:** A Python-based offline Ant Colony Optimization (ACO) solver that simulates and computes the search-based near-optimal schedules (both preemptive and non-preemptive baselines).

To reproduce the experimental results, researchers can utilize the following workflows provided in the repository:

- 1) **Manual Embedded Build:** Navigate to `codes/ProjectContextAwareScheduling`, configure the CMake preset with the desired scheduler (e.g., `cmake --preset Release -DACTIVE_SCHEDULER=CONTEXT_AWARE`), and build the binary to flash manually.
- 2) **Automated CLI Benchmarks:** Run the Python automation script (`python`

`app/automate_schedulers.py --port COM4 --baud 115200 --duration 30`) to sequentially configure, build, flash, and record UART telemetry logs for the Native, EDF, and Context-Aware schedulers.

- 3) **Web GUI Dashboard:** Launch the server (`python app/gui_server.py`) and access the dashboard at `http://127.0.0.1:8000` to interactively tune parameters, trigger flashes, and visualize hardware telemetry in real time.
- 4) **Offline ACO Solver:** Execute `python -m offline_aco_baseline.main` to run the offline simulator and generate convergence graphs without requiring hardware in the loop.

IX. CONCLUSION AND FUTURE WORK

This project successfully addressed the critical challenge of shared memory contention in mixed-criticality, real-time embedded systems, a phenomenon that degrades timing predictability and threatens determinism on single-core uniprocessor architectures. We developed and evaluated a lightweight, user-space context-aware scheduling wrapper (CAS) on an STM32L4-series microcontroller running FreeRTOS. By extending the classical Earliest Deadline First (EDF) algorithm with a dynamic, slack-aware heuristic, our scheduler evaluates the memory intensity of pending tasks at runtime. It reorders execution sequences to safely separate consecutive memory-heavy operations without violating hard real-time deadlines.

To validate our online heuristic, we paired it with a search-based offline Ant Colony Optimization (ACO) reference model. The ACO solver complements the embedded evaluation by generating near-optimal job orderings and preemptive segment-level dispatch traces for the same task models. Experimental evaluations across a four-task, mixed-criticality workload ($\tau_1, \tau_2, \tau_3, \tau_4$) demonstrated that both the baseline EDF and the proposed CAS drastically outperform the native FreeRTOS static-priority scheduler, which completely starved low-priority and memory-bound tasks (τ_3 and τ_4) under intense memory pressure. The ACO experiments show that offline global search is useful for identifying workload regimes where schedule ordering has a strong effect on lateness and memory-transition cost, while the proposed CAS provides a practical online approximation suitable for FreeRTOS deployment.

Crucially, while CAS and EDF yielded virtually identical performance regarding final Deadline Miss Ratios (DMR) and task jitter, the Context-Aware Scheduler succeeded in its primary objective: it reduced the total memory contention penalty by more than half compared to pure EDF. Although introducing runtime context awareness predictably increased the average scheduler decision overhead, the substantial reduction in memory contention penalties heavily outweighed these additional computation cycles.

Despite these positive outcomes, the current methodology revealed certain limitations. Most notably, extracting metrics via the USART2 peripheral during benchmarking induced

a hardware "observer effect". The UART interrupts stalled the processor pipeline and competed for the exact hardware resources utilized by the tasks, injecting an artificial layer of hardware contention into the results. Additionally, the system currently models memory intensity (M_i) as a static, task-level constant, which fails to capture fine-grained intra-task phase variations.

Future work will focus on refining the evaluation framework and expanding the scheduler's capabilities. Key areas for extension include:

- **Hardware-Assisted Telemetry:** Transitioning from active UART logging to zero-stall hardware debugging channels, such as the ARM Instrumentation Trace Macro-cell (ITM) / Serial Wire Output (SWO), to eliminate the observer effect entirely.
- **Algorithmic Optimization:** Refining the CAS queue-reordering algorithm to achieve $O(1)$ constant-time complexity using bitmap-based logic, ensuring the scheduler decision overhead remains strictly bounded even as the task set scales.
- **Multi-Core Scalability:** Extending the context-aware scheduling heuristic to support multi-core embedded architectures. This involves transitioning from mitigating sequential preemption penalties to dynamically managing concurrent memory bus contention and shared cache interference across parallel execution units.

ACKNOWLEDGMENTS

This study was conducted within the scope of the graduate course CMP720 Embedded System Design at the Department of Computer Engineering, Hacettepe University. We would like to express our sincere gratitude to Asst. Prof. Dr. Selma Dilek for her invaluable guidance throughout this study. Her insightful feedback and expertise were instrumental in shaping this project.

REFERENCES

- [1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [2] H. Yun, G. Yao, R. Pellizzoni, M. Caccamo, and L. Sha, "Memguard: Memory bandwidth reservation system for efficient performance isolation in multi-core platforms," in *Proceedings of the IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2013, pp. 55–64.
- [3] H. Yun, R. Mancuso, Z.-P. Wu, and R. Pellizzoni, "Pallocc: Dram bank-aware memory allocator for performance isolation on multicore platforms," in *Proceedings of the IEEE 20th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2014, pp. 155–166.
- [4] R. Pellizzoni, E. Betti, S. Bak, G. Yao, J. Criswell, M. Caccamo, and R. Kegley, "A predictable execution model for cots-based embedded systems," in *Proceedings of the IEEE 17th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2011, pp. 269–279.
- [5] J. M. Rivas, J. Goossens, X. Poczekajlo, and A. Paolillo, "Implementation of memory centric scheduling for cots multi-core real-time systems," in *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 133, 2019, pp. 7:1–7:23.
- [6] G. Schwärzke, T. Kloda, G. Gracioli, M. Bertogna, and M. Caccamo, "Fixed-priority memory-centric scheduler for cots-based multiprocessors," in *32nd Euromicro Conference on Real-Time Systems (ECRTS 2020)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 165, 2020, pp. 1:1–1:24.

- [7] W. Lunniss, S. Altmeyer, C. Maiza, and R. I. Davis, "Integrating cache related pre-emption delay analysis into edf scheduling," in *Proceedings of the IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2013, pp. 75–84.
- [8] C. Lin and S. A. Brandt, "Improving soft real-time performance through better slack reclaiming," in *Proceedings of the 26th IEEE International Real-Time Systems Symposium (RTSS)*, 2005, pp. 410–421.
- [9] G. Phavorin, P. Richard, J. Goossens, C. Maiza, L. George, and T. Chapeaux, "Online and offline scheduling with cache-related preemption delays," *Real-Time Systems*, vol. 54, no. 3, pp. 662–699, 2018.